```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m29.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m29.c -o m29
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m29
==========================================
          APPEND REDIRECTION
==========================================
Enter output filename: m15
Enter message to append: NIce bro

Data appended successfully.
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024

int main()
{
    char filename[MAX_INPUT];
    char message[MAX_INPUT];

    int fd;
    pid_t pid;

    printf("==========================================\n");
    printf("          APPEND REDIRECTION\n");
    printf("==========================================\n");

    printf("Enter output filename: ");

    if (fgets(filename,
              sizeof(filename),
              stdin) == NULL)
    {
        return 1;
    }

    filename[strcspn(filename, "\n")] = '\0';

    printf("Enter message to append: ");

    if (fgets(message,
              sizeof(message),
              stdin) == NULL)
    {
        return 1;
    }

    message[strcspn(message, "\n")] = '\0';

    /*
     * O_APPEND ensures that every write
     * occurs at the end of the file.
```

```
    printf("Enter output filename: ");

    if (fgets(filename,
              sizeof(filename),
              stdin) == NULL)
    {
        return 1;
    }

    filename[strcspn(filename, "\n")] = '\0';

    printf("Enter message to append: ");

    if (fgets(message,
              sizeof(message),
              stdin) == NULL)
    {
        return 1;
    }

    message[strcspn(message, "\n")] = '\0';

    /*
     * O_APPEND ensures that every write
     * occurs at the end of the file.
     *
     * Existing data is preserved.
     */
    fd = open(
        filename,
        O_WRONLY | O_CREAT | O_APPEND,
        0644
    );

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    pid = fork();

    if (pid < 0)
```

```
    {
        perror("open");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Redirect stdout to append file.
         */
        if (dup2(fd, STDOUT_FILENO) == -1)
        {
            perror("dup2");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);

        printf("%s\n", message);

        fflush(stdout);

        exit(EXIT_SUCCESS);
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("\nData appended successfully.\n");

    return 0;
}
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m30.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m30.c -o m30
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m30
==========================================
          STDERR REDIRECTION
==========================================
Enter error output filename: odu

Error redirection completed.
Check the specified error file.
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```
    fd = open(
        filename,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
    {
        perror("open error file");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Redirect standard error (file descriptor 2)
         * to the file.
         */
        if (dup2(fd, STDERR_FILENO) == -1)
        {
            perror("dup2");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);

        /*
         * Generate an error.
         */
        FILE *file = fopen(
            "file_that_does_not_exist.txt",
            "r"
```

```
        /*
         * Redirect standard error (file descriptor 2)
         * to the file.
         */
        if (dup2(fd, STDERR_FILENO) == -1)
        {
            perror("dup2");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);

        /*
         * Generate an error.
         */
        FILE *file = fopen(
            "file_that_does_not_exist.txt",
            "r"
        );

        if (file == NULL)
        {
            perror("ERROR");
        }

        /*
         * Generate another error.
         */
        int test_fd = open(
            "/invalid/path/test.txt",
            O_RDONLY
        );

        if (test_fd == -1)
        {
            perror("open");
        }

        exit(EXIT_FAILURE);
    }

    close(fd);

    waitpid(pid, NULL, 0);
```

```
        FILE *file = fopen(
            "file_that_does_not_exist.txt",
            "r"
        );

        if (file == NULL)
        {
            perror("ERROR");
        }

        /*
         * Generate another error.
         */
        int test_fd = open(
            "/invalid/path/test.txt",
            O_RDONLY
        );

        if (test_fd == -1)
        {
            perror("open");
        }

        exit(EXIT_FAILURE);
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("\nError redirection completed.\n");
    printf("Check the specified error file.\n");

    return 0;
}
```